

trailing “excess” elements will be assigned some encountered runtime values from memory.

In case of array of char, you can use a shorter string literal notation. For example:

```
/* The two declarations are identical: */
const char msg1[] = {'T', 'e', 's', 't', '\0'};
const char msg2[] = "Test";
```

Arrays in Expressions

When name of the array comes up in expression evaluation (except with operators & and sizeof), it is implicitly converted to the pointer pointing to array’s first element.

Multi-dimensional Arrays

An array is one-dimensional if it is of scalar type. One-dimensional arrays are sometimes referred to as vectors. Multidimensional arrays are constructed by declaring arrays of array type. These arrays are stored in memory in such way that the right most subscript changes fastest, i.e. arrays are stored “in rows”. Here is a sample 2-dimensional array:

```
float m[50][20]; /* 2-dimensional array of size 50x20
*/
```

Variable m is an array of 50 elements, which in turn are arrays of 20 floats each. Thus, we have a matrix of 50x20 elements: the first element is m[0][0], the last one is m[49][19]. First element of the 5th row would be m[0][5].

If you are not initializing the array in the declaration, you can omit the first dimension of multi-dimensional array. In that case, array is located elsewhere, e.g. in another file. This is a commonly used technique when passing arrays as function parameters:

```
int a[3][2][4]; /* 3-dimensional array of size 3x2x4 */
voidfunc(int n[][2][4]) { /* we can omit first dimension
*/
    //...
    n[2][1][3]++; /* increment the last element*/
} //~
void main() {
    //...
    func(a);
```